



Inspiring Excellence

CSE422: ARTIFICIAL INTELLIGENCE

Software Quality Classification

GROUP 04

Member 1:

Nafiz Shahriar Sami
23201336

Member 2:

Afnan Mohammad Hafiz
23241004

TABLE OF CONTENTS

	<u>Pages</u>
● Introduction —————	3
● Dataset description —————	3
● Imbalanced dataset—————	5
● Exploratory Data Analysis—————	6
● Dataset preprocessing—————	7
● Dataset splitting —————	7
● Model training and testing (supervised)—————	8
● Model selection/Comparison analysis—————	12
● Conclusion—————	19

Introduction

Software quality is a critical factor that directly influences its reliability, maintainability, and overall user satisfaction. In modern software development environments, codebases change constantly, making it inefficient for developers to find errors and debug them. This process is time-consuming and redundant in utilizing the efficiency of developers, also making it prone to error. This project aims to build a machine learning classification model to predict the quality label of a software module, categorized as High, Medium, or Low, based on measurable code attributes and developer-related features.

By learning patterns from historical software metrics data, the model seeks to assist teams working to identify high-risk sections. The problem is categorized as a multi-class classification task, where the target variable takes one of three discrete quality labels. The motivation lies in collecting data-driven approaches to reduce defect rates and support decision-making that will make the model more efficient and accurate in identifying patterns and getting accurate predictions and outcomes

Dataset description

- 9 Features in total
- It is a classification problem. This project is framed as a classification problem because the target variable to be predicted, `Quality_Label`, is categorical rather than continuous. In machine learning, when the target output takes on discrete labels or classes, the task is to classify input data into one of these predefined categories. The target classes to be predicted are high, medium, and low.
- There are 1600 datapoints.

- Both quantitative and categorical features are present. There are 7 quantitative features and 2 categorical features.
- Yes, we are supposed to encode the categorical variables. Most machine learning algorithms require numerical input. They cannot interpret raw text or categorical labels.
- Correlation Matrix:

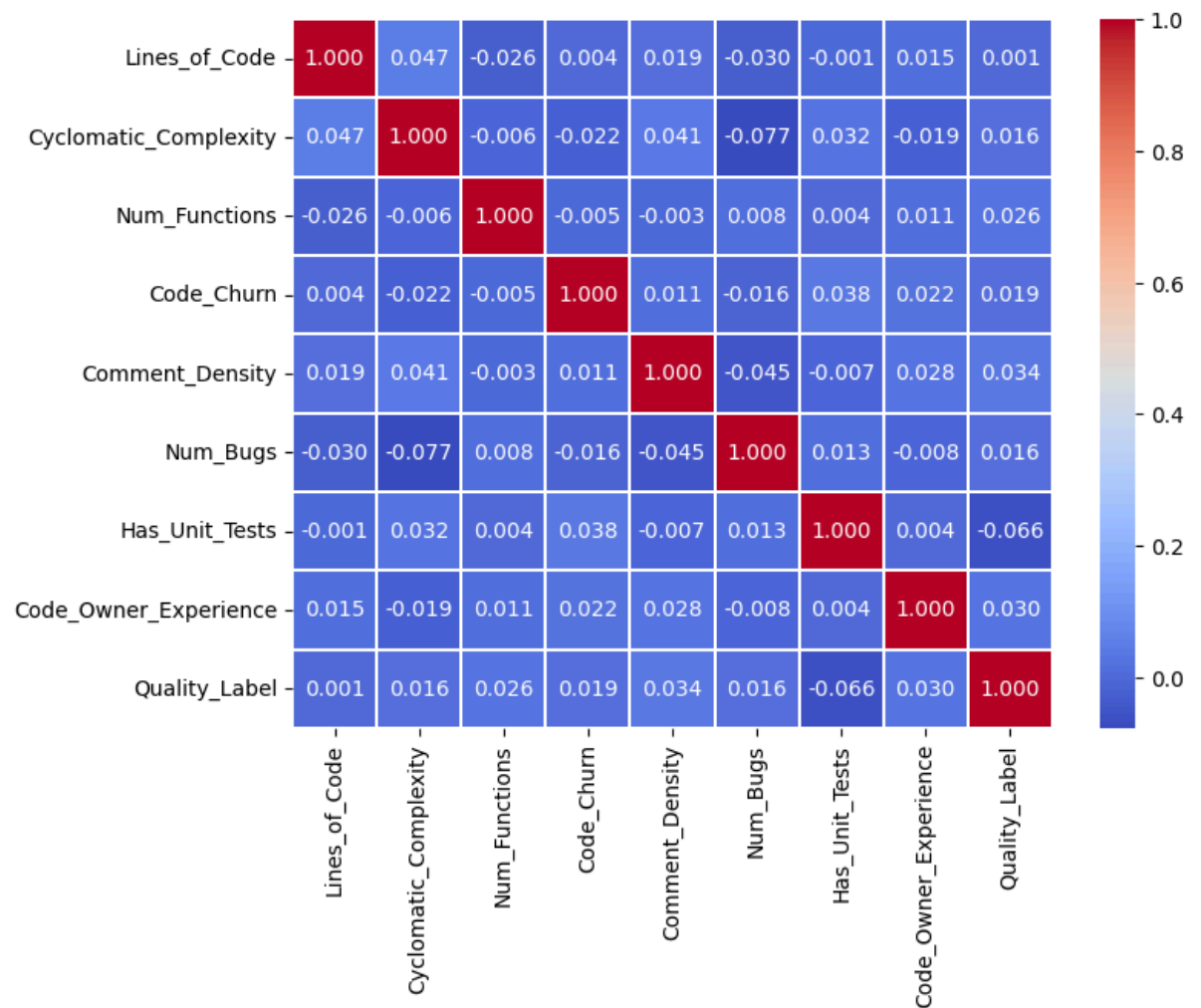


Figure 1a: Correlation of all the features using the seaborn library

-After the correlation test, almost no meaningful correlations exist between any features.

Imbalanced Dataset

For the output feature, all unique classes do not have an equal number of instances.

Quality Label

High	Medium	Low
566	501	533

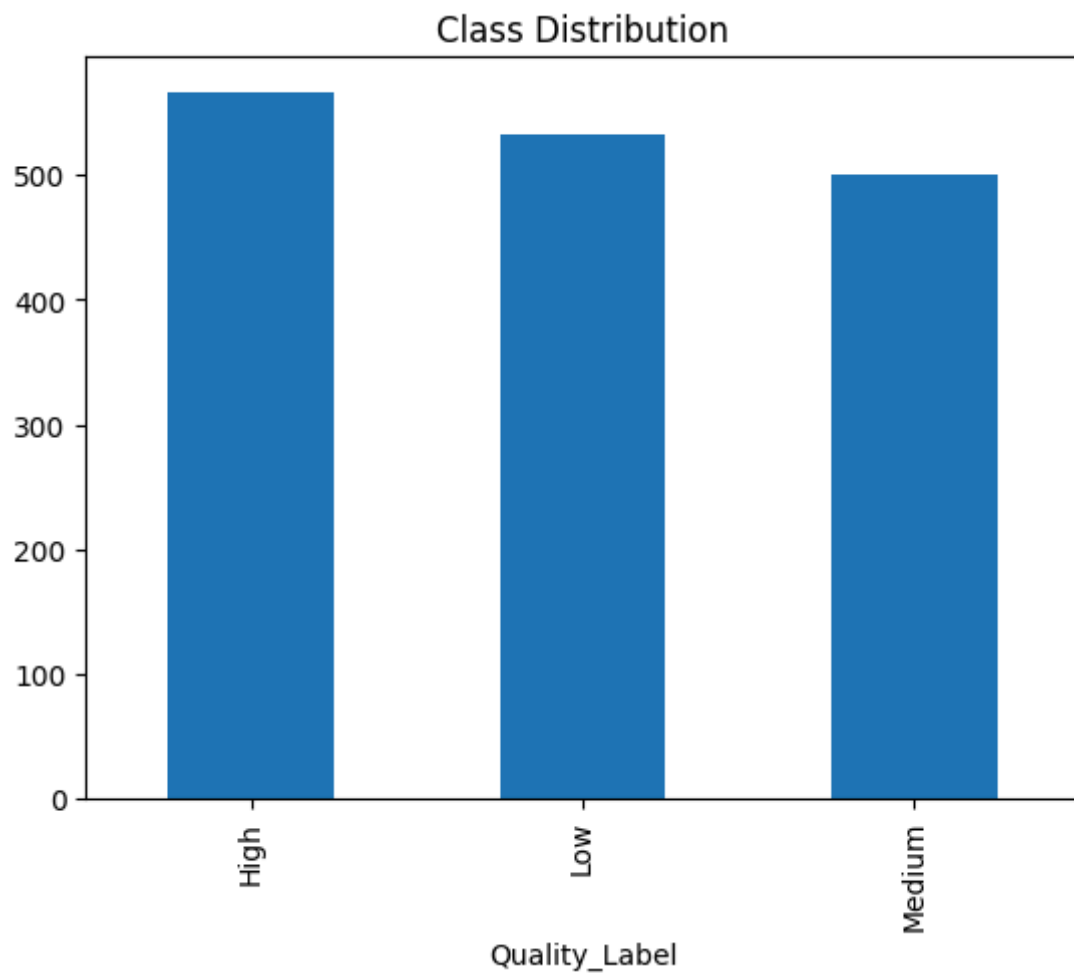


Figure 1b: Representing imbalance using a bar chart with 3 classes

Exploratory Data Analysis

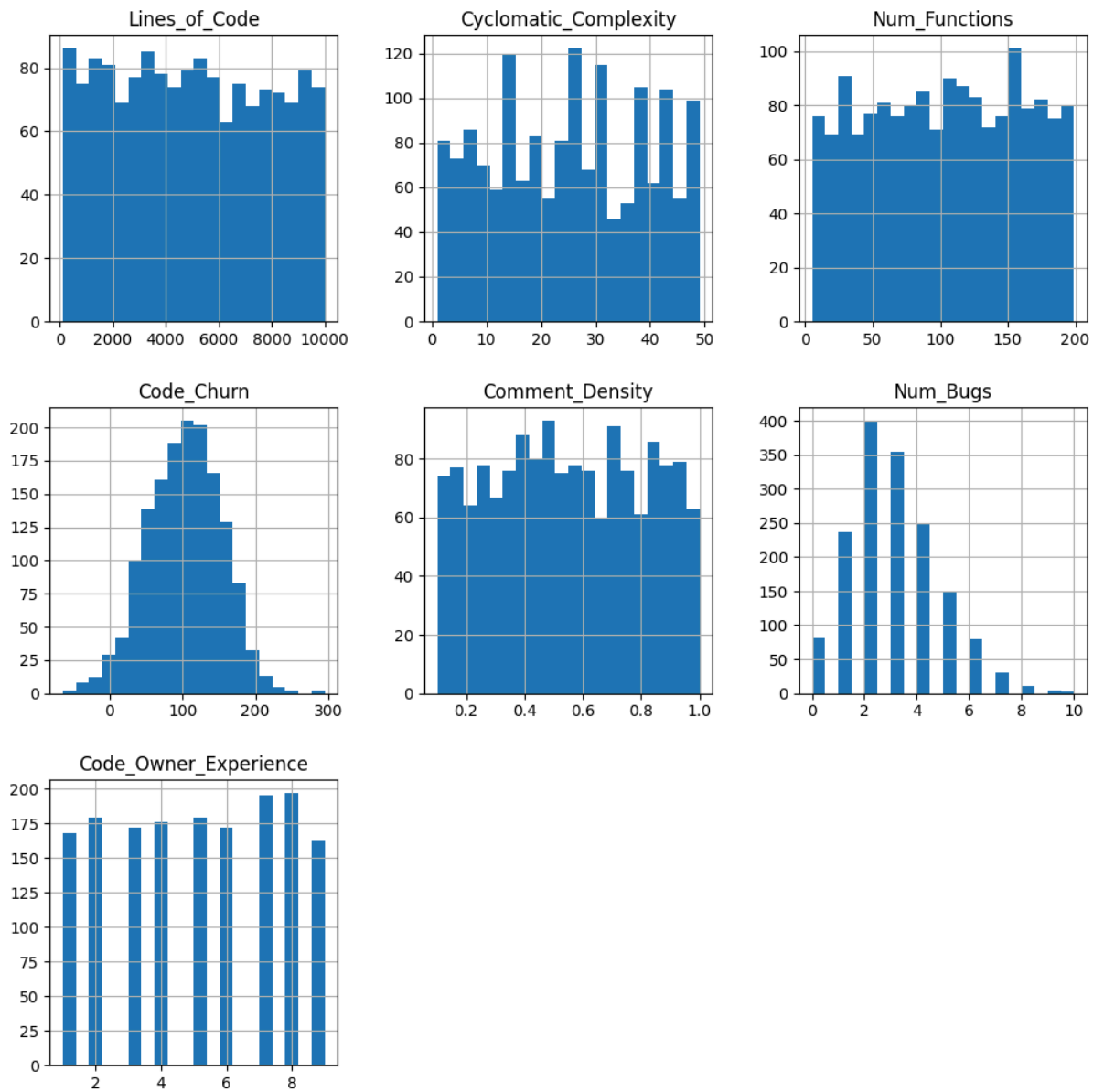


Figure 1c: Histogram plot

Dataset preprocessing

Fault	Solution	Reasoning
Null/Missing values	Replaced (229 found)	The dataset had a few missing values that were replaced with the mean (when numerical) and the most frequent (when categorical)
Duplicate values	Checked (0 found)	The dataset does not have any duplicate values; no imputation was required
Categorical value	Label encoding and one-hot encoding	Converted text (Has_Unit_Tests) to numerical/binary formats for model compatibility.
Feature scaling	StandardScaler	Used to normalize numerical features.

Dataset splitting

The data was split into an 80% training set to help the model learn patterns and a 20% test set to evaluate how well it performs on new, unseen information.

Model training & testing (Supervised)

We implemented and evaluated the following models:

- **Logistic Regression:** Used as a baseline linear classifier.

Logistic Regression Accuracy: 0.3125				
	precision	recall	f1-score	support
High	0.39	0.43	0.41	127
Low	0.24	0.22	0.23	99
Medium	0.26	0.24	0.25	94
accuracy			0.31	320
macro avg	0.30	0.30	0.30	320
weighted avg	0.31	0.31	0.31	320

Figure 2a: Accuracy and metrics of logistics regression

- **Neural Network:** A 2-layer MLP architecture (64, 32 neurons) was applied to capture the complicated deep patterns.

Neural Network Accuracy: 0.328125				
	precision	recall	f1-score	support
High	0.39	0.32	0.35	127
Low	0.33	0.39	0.36	99
Medium	0.26	0.27	0.26	94
accuracy			0.33	320
macro avg	0.33	0.33	0.32	320
weighted avg	0.33	0.33	0.33	320

Figure 2b: Accuracy and metrics of the neural network

- **Decision Tree:** Used for its ability to handle non-linear relationships. Known for its highly interpretable, easy-to-visualize, and minimal data scaling qualities.

```

Decision Tree Accuracy: 0.3375

Classification Report:

```

	precision	recall	f1-score	support
High	0.38	0.41	0.39	127
Low	0.31	0.57	0.40	99
Medium	0.00	0.00	0.00	94
accuracy			0.34	320
macro avg	0.23	0.33	0.27	320
weighted avg	0.25	0.34	0.28	320

Figure 2c: Accuracy and metrics of the decision tree

[Unsupervised] For unsupervised learning problems, we used K-means. K-means is an unsupervised machine learning algorithm that groups data points into k clusters based on similarity. It is used to segment data and simplify large data sets by organizing them into clusters. The cluster visualization was done by PCA. The figure shown below contains a fixed k=3 value, which indicates the 3 classes for the target feature.

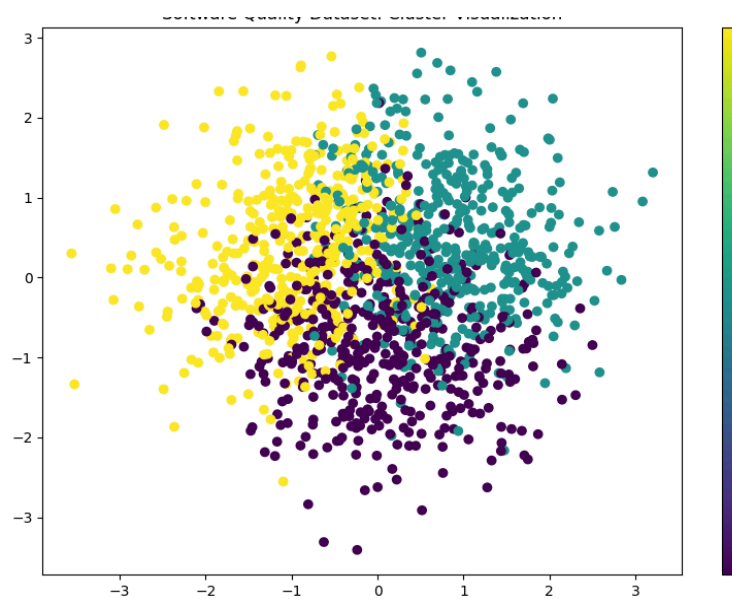


Figure 2d: K-means clustering with k=3

The **elbow graph** is used to find the **optimal number of clusters (k)** for K-Means clustering. Because the dataset has very low correlation overall and is randomly generated, the elbow graph shown below does not have a distinct point (elbow) which indicates the number of clusters. Based on this, the newly generated k-means clustering graph is also shown below with no predefined cluster value.

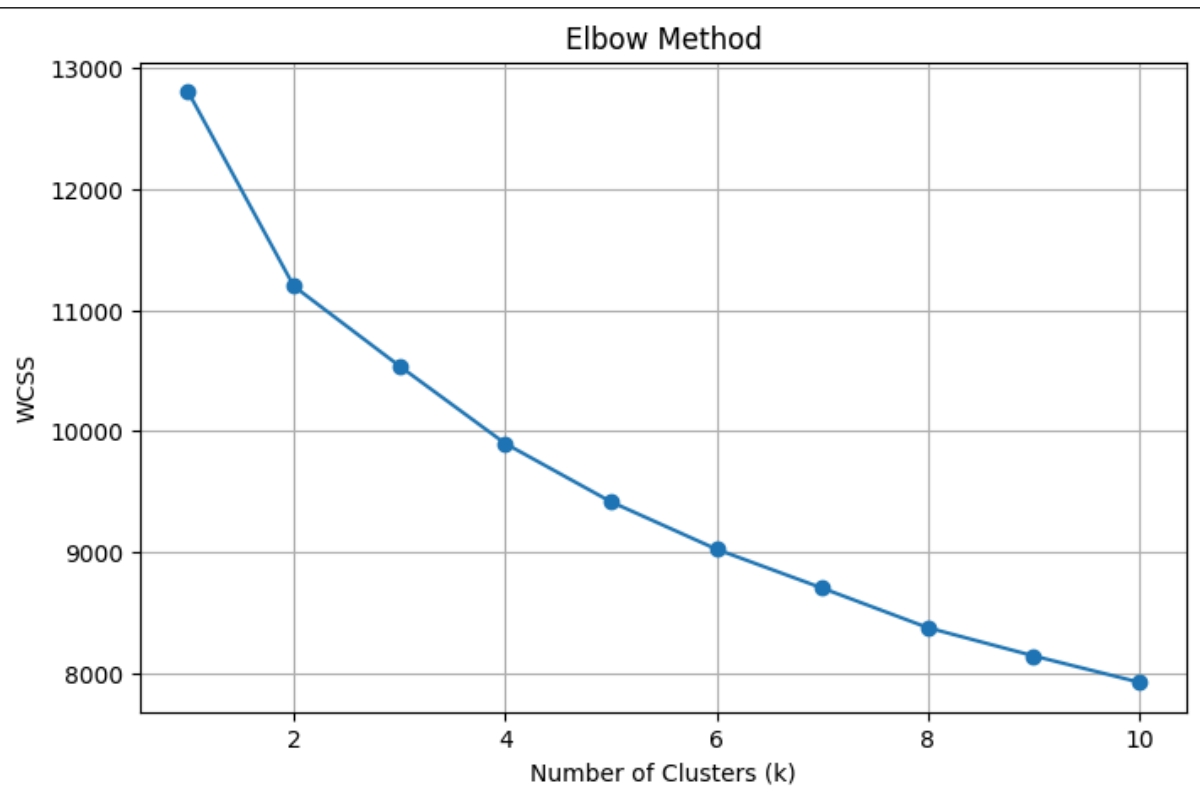


Figure 2e: Elbow Graph

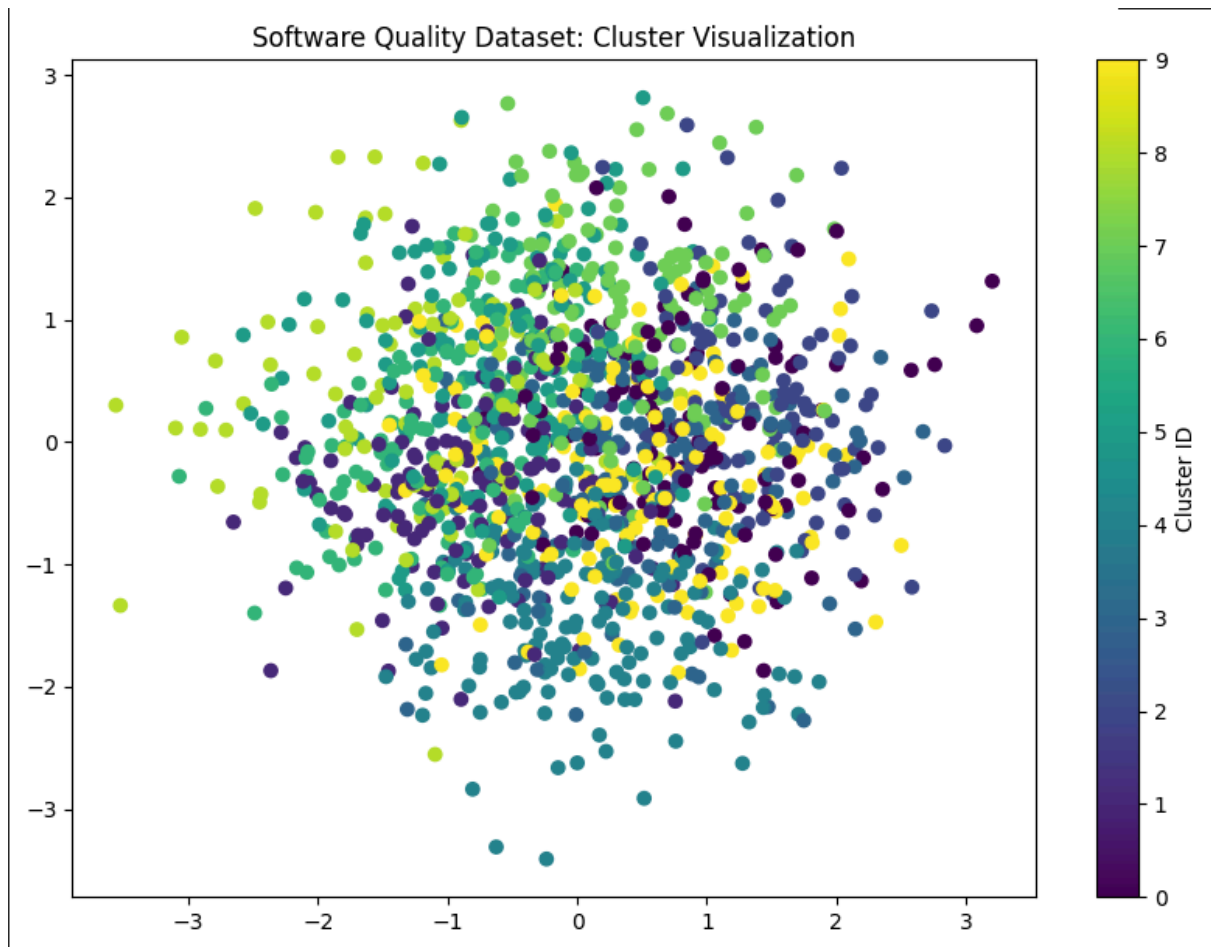


Figure 2f: K-means clustering without predefined clusters

Model selection/Comparison analysis

Bar chart showcasing the prediction accuracy of all models:

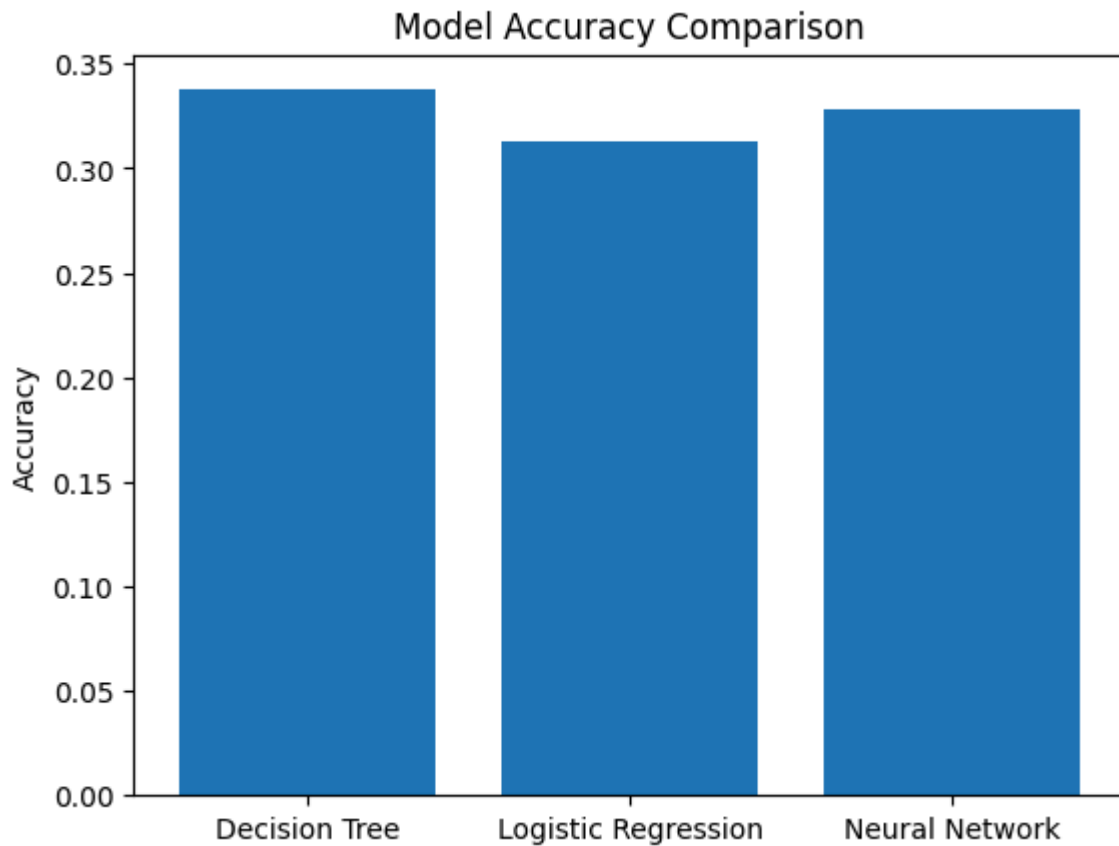


Figure 3a: Bar Chart Comparison of Model Accuracy

1. Neural network: Accuracy is 0.328125 (32.28%). A neural network model is used to model deep, complex, and nonlinear interactions across the dataset using a 2-layer architecture. The key merit of this model is that it handles complex, non-linear patterns within multi-class features effectively. But it lacks when it acts as a black box, thus offering less interpretability.

2. Decision Tree Classifier: Accuracy is 0.3375 (33.75%). The decision tree model is used to capture non-linear relationships and hierarchical rules among logistics features. The main merit of this model is that it is highly interpretable, easy to visualize, and requires minimal data preprocessing/scaling. Its demerits lie in overfitting if not tuned properly, and it is sensitive to minor variations in the data samples.

3. Logistic Regression: Accuracy is 0.3125 (31.25%). A logistic regression model is used as a baseline linear model. Its primary merit is that it is computationally inexpensive, simple to implement, and provides a clear baseline for performance. However, it assumes a linear relationship, which fails to capture complex interactions and tends to predict the majority class, thus resulting in poor identification of minority classes.

In this experiment, the models we have used provided low accuracy rates, with decision tree, logistic regression, and neural network achieving around 33.75%, 31.25%, and 32.28%, respectively. Thus, it indicates that the models do not learn meaningful patterns from the dataset but rather guess randomly.

Precision and recall comparison of each model

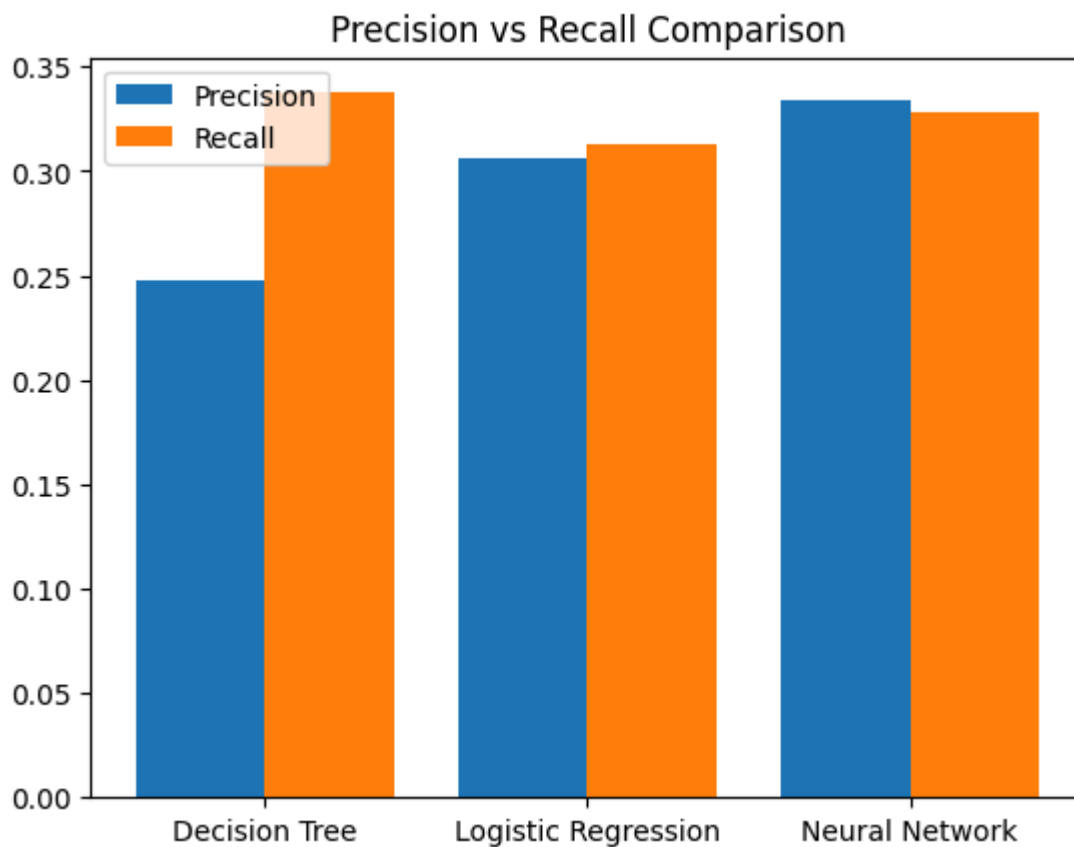


Figure 3b: Precision vs Recall Comparison using a bar chart

Precision and recall were used to evaluate the classification performance of each model. The decision tree achieved moderate results, whereas logistic regression showed the least performance. In contrast, the neural network exhibits slightly better precision and recall values across all classes but does not ensure reliability due to its low accuracy.

Confusion Matrix

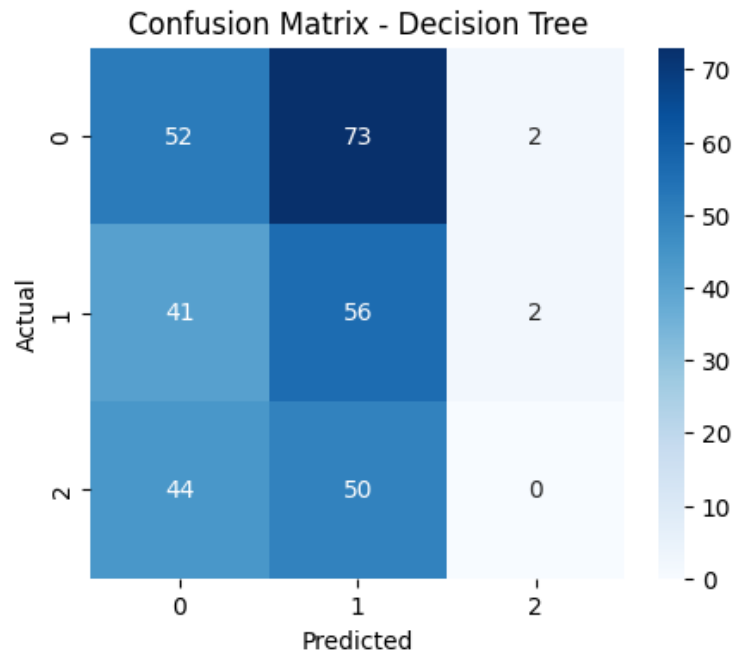


Figure 3c: Confusion Matrix of Decision Tree

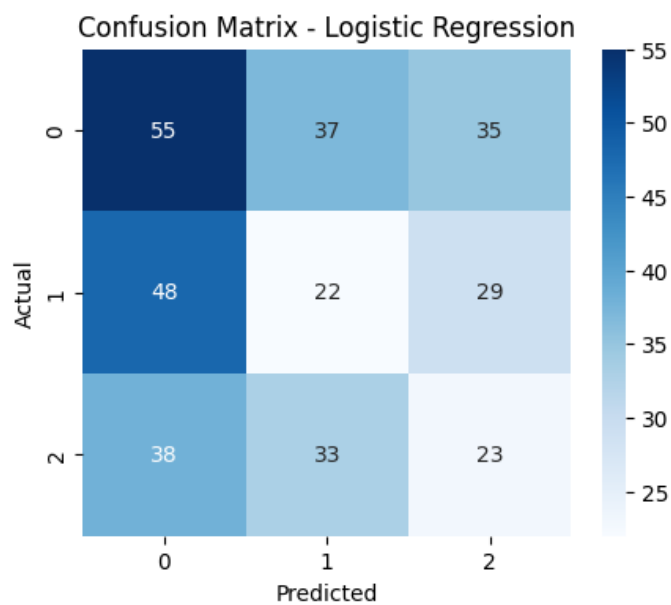


Figure 3d: Confusion Matrix of Logistic Regression

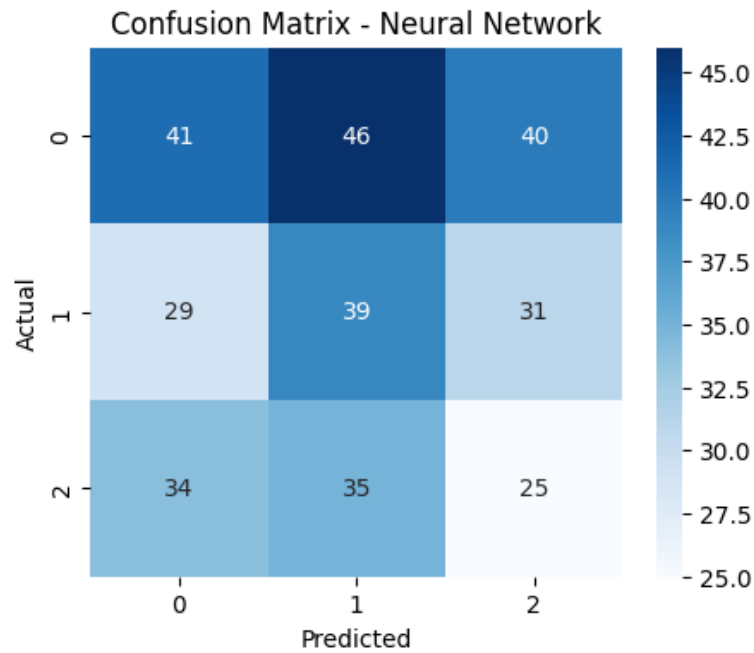


Figure 3f: Confusion Matrix of Neural Network

The confusion matrices for all three models reveal a consistent pattern of poor class discrimination across the high, medium, and low quality labels, which aligns with the earlier findings of near-random accuracy

In the decision tree confusion matrix (Figure 3c), while the model correctly identifies parts of instances from each class, there is still a vast number of predictions that are scattered across the three labels. The model does not favor one class over another, as miscalculations are not defined in one particular direction. This shows that even though the model tries to figure out a pattern from the given data, it cannot construct a decision boundary that will separate the three modules of high, medium, and low. Meanwhile, the diagonal values represent more accurate predictions than those of off-diagonal values, which highlights the model's limited confidence across all three classes.

The logistic regression confusion matrix (Figure 3d) highlights the prediction of one dominant class over the others. This is seen as a known limitation of linear models, which is depicted when decision boundaries between classes are not linearly separable. Moreover, the model peaks toward the class with a slightly higher occurrence in the training data, leading to

incorrect classifications, treating it as high rather than medium or low. This bias results in the model underperforming on the minority classes, resulting in a diminishing of the accuracy results

The neural network confusion matrix (Figure 3f) spreads predictions relatively and uniformly across all the classes of high, medium, and low due to its evenly distributed misclassifications. However, the model fails to learn any discriminating pattern at all, even though seeming less biased. Given the small dataset size of 1,600 samples and the weak feature-to-label correlations observed during EDA, the 2-layer MLP architecture has less data to effectively train its parameters and achieve accurate results. Thus, it leads to a confusion matrix in which no single class is predicted with accuracy, producing near-random outputs for most test instances

In conclusion, the confusion matrix analysis confirms that all three models that are being run face equal struggle to classify software quality. The main cause is the absence of strong discriminating signals in the feature set, found during exploratory data analysis, which indicates near-zero correlations. Improving the quality of the input features and data can impact getting better class separation across all models that are being run

AUC score for each model

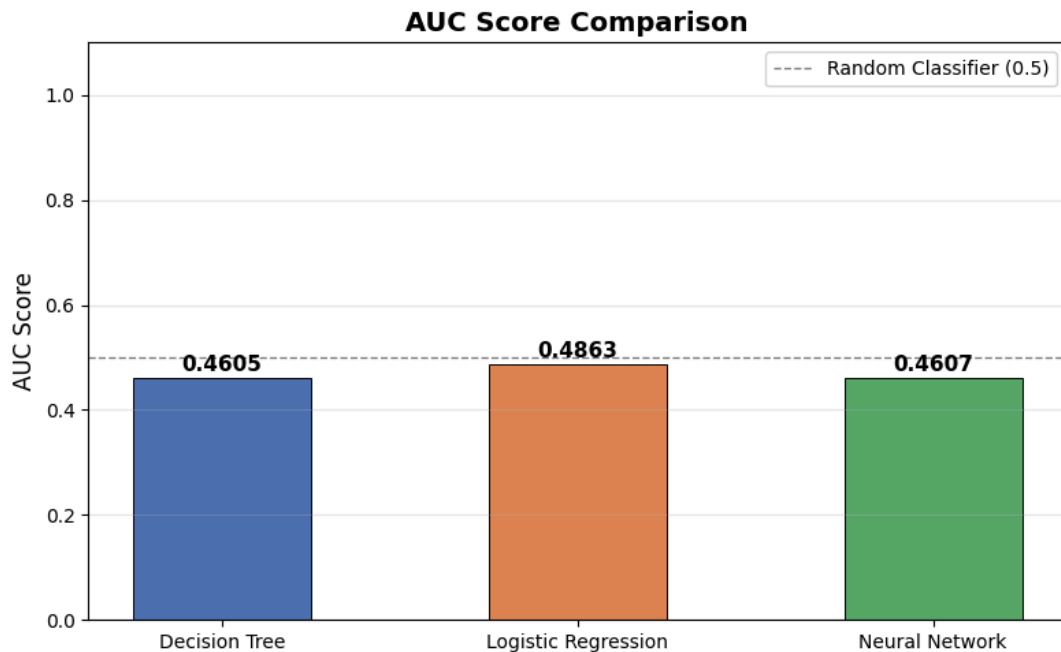


Figure 3g: AUC Score comparison using a bar chart

The AUC (Area Under the Curve) highlights the ability to distinguish between the three quality classes. All three models recorded AUC values close to 0.5, maintaining consistency with the low accuracy results viewed earlier. This indicates that the models cannot distinguish between the three classes. The logistic regression recorded the highest AUC among the three, followed by the neural network, while the decision tree recorded the lowest. However, the differences are very close to each other, indicating values being close to a random-guessing performance. It is evident that the dataset does not have sufficient signal for any of the tested models so that they can differentiate the classes, thus drawing conclusions from the confusion matrix analyses and their accuracy.

ROC curve for each model

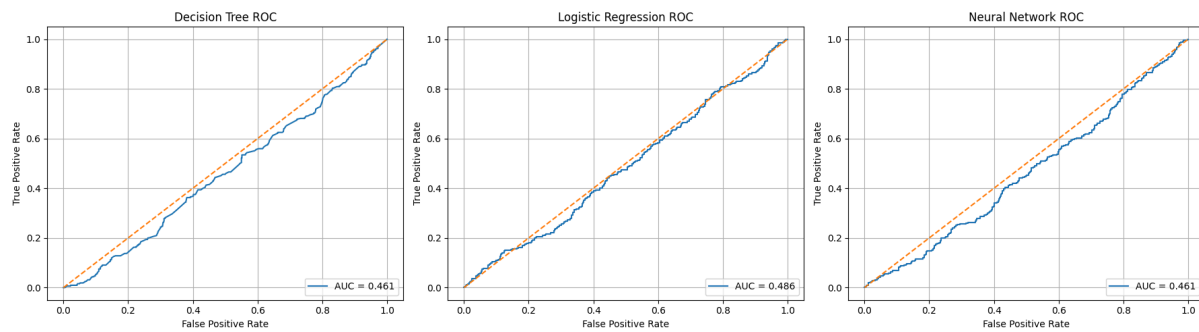


Figure 3h: ROC curve of each model

The ROC curves were plotted using a one-vs-rest strategy for each of the three quality classes. The curve lies close to the diagonal reference line, representing random guessing across all three models. The AUC values and low accuracy rates reported earlier are consistent with this case. None of the three models shows a curve toward the top-left corner, indicating true positive rates remain low relative to false positive rates across all class thresholds. Compared to logistic regression and the neural network, the decision tree produces a curve that is slightly further from the diagonal, making it the best-performing model in the field of class discrimination.

Meanwhile, the logistic regression shows the least separation from the diagonal, reflecting its difficulty in capturing the non-linear structure of the data. On the whole, the ROC analysis confirms that the models struggle to classify software quality for the given dataset. To achieve meaningful separation between classes, further improvement in feature design or even model tuning might be needed.

Conclusion

From the result, we understood that the dataset had little to no correlation between the target feature (y) and other features (X). For example, the chance of either getting a low-quality label or a high-quality label is 0.33 because the result is being generated randomly. This is the reason why we are getting such low accuracy for all of the supervised models. The low performance or low accuracy collected from all the models was expected, as there was no correlation. If we had collected data that had better correlation or target features that were not generated randomly, theoretically, we could have had a better outcome.

Furthermore, the most significant challenge we faced was the low accuracy. We were unsure of the validity of the models, the dataset, and our preprocessing. We cross-checked each section and consulted our course supervisors. Firstly, we had dropped the missing values, but afterwards we replaced them and tried again. Finally, to test whether getting such low accuracy was accurate or not, we implemented the “Adam Optimizer.” The output generated from this procedure is shown visually.

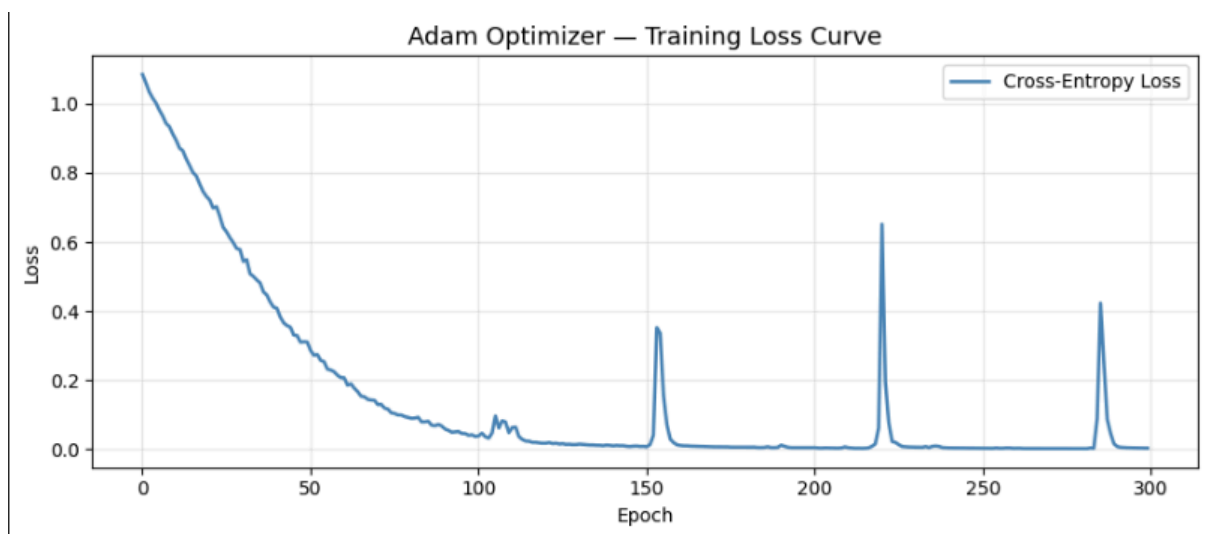


Figure 4a: Adam Optimizer

To conclude, the ML project based on the Software Quality Dataset had a few issues regarding randomness and dataset correlation, but it could be improved using a newly collected dataset with better dependency and balance.